A Danger-Field PPO Controller for the Dodge Arcade Game: Hard-Difficulty Evaluation, Decision Frequency, and Where Games End

Lucas Starkey

***Abstract*—We report the current state of a reinforcement-learning agent for Dodge, a small score-driven arcade game whose hazards intensify as the player scores. Training and evaluation run on Turner's native Rust port of the cartridge. The policy, VelocityFlow v2, is a 111k-parameter convolutional network whose only output is a time-sliced danger field over the arena; a fixed controller reads that field along the simulated path of each of the nine actions, so the logits are path danger and nothing between field and action is learned. After 24,000 PPO updates (91.6M decisions), the best checkpoint survives a mean of 8,888 alive frames (about 2.5 minutes) on the hardest setting, with a worst-quarter mean of 2,902, on 1,024 evaluation seeds with no frame cap. It beats an earlier checkpoint on the same seeds by 5,477 frames (95% interval +5,069 to +5,920). Evaluating it with a decision every fourth frame instead of every frame cuts survival by 64% (−5,647 frames), the reverse of a recent report's "control-frequency surprise" and consistent with our checkpoint having been trained on per-frame decisions; sampling actions instead of taking the argmax costs 2,818 frames. Death telemetry shows that the anti-camping guard fires in 4% of games, that 69% of deaths occur while a pattern is active (in 42% of frames), and that 62% of enemy-contact deaths happen during patterns. Replaying the 496 deaths inside static patterns shows that the deadliest layouts kill mostly through enemies that catch the player in the open or against a block, and through rectangles only where the layout is a dense grid or a frame of bars. We describe the game, model, and protocol, report these results with their limits, and pose specific questions.**

***Index Terms*—reinforcement learning, proximal policy optimization, potential fields, convolutional networks, action repeat, evaluation methodology, arcade games**

I. Introduction

DODGE is a short arcade game in which the player steers a square around a small arena, avoids moving enemies and large rectangular hazards, and collects pickups that raise the score. Scoring makes the game faster and more dangerous, so the objective that looks natural, a high score, conflicts with the one that keeps the player alive.

This is a working draft written to solicit feedback. Results are exploratory, come from a single training lineage, and have not been peer reviewed.

This draft follows the evaluation protocol of a recent working draft on the same game by Turner [1], which trained an entity-transformer teacher on a Rust port, so that the two efforts can be compared on like terms: alive frames, quarter means, sampled versus argmax actions, one- versus four-frame decisions, and instrumented death telemetry. The two projects now share that simulator but differ in almost everything else: observation, architecture, reward, difficulty, and training history. Cross-paper comparisons of levels are therefore indicative only (Section VII).

We do not present a finished or competitive agent. The contributions we think are of interest are:

1) A field-only policy: a convolutional network that predicts danger per arena cell at six future times, read by a fixed path-sampling controller, so that the network never outputs an action directly (Section IV).
2) A paired, uncapped, 1,024-seed evaluation of the best checkpoint on the hardest setting, with bootstrap intervals and an earlier checkpoint and two baselines on the same seeds, drawn with the training dashboard's own evaluation charts (Section VI-A).
3) A decision-frequency result that runs opposite to [1]: coarser decisions cut survival by nearly two thirds for a policy trained at one decision per frame (Section VI-B).
4) Death telemetry: the anti-camping guard is nearly absent from this policy's games, while patterns, and enemies during patterns, account for most deaths; per-pattern lethality from at least 35 encounters each; and replayed paths into the deadliest static patterns (Sections VI-C–VI-E).

Section VIII lists the questions on which we most want input.

II. The Game

Dodge is a PICO-8 cartridge by Artridge [2]. The playfield is a 128×128 pixel square updated at 60 Hz. The player moves in eight directions or stays still. Movement has inertia: each frame velocity is multiplied by 0.8 and each held direction adds 0.5, so the effect of an action depends on current motion.

Hazards. Enemies enter the arena, home on the player, grow, and collide with one another. Kamikaze enemies

leave a growing explosion. Pattern events add large rectangles that are telegraphed before becoming lethal: static patterns, whose rectangles hold or morph in place, and moving patterns, which sweep across the field. Some pattern geometry extends past the visible screen.

Pickups and score. Pickups explode nearby enemies, freeze the game, or shrink the player; score rises when enemies die and when pickups are collected. Game speed, enemy spawn timing, and the pool of eligible patterns depend on score.

Anti-camping guard. If the player stays inside a small central region for more than 50 consecutive frames, the cartridge routine `fyou` spawns an enemy at every spawn point.

Death. A game ends on the first lethal contact. We record its class: normal enemy, kamikaze (body or explosion), static-pattern rectangle, or moving-pattern rectangle.

III. Environment and Observations

Simulator. All training and evaluation now run on Turner's native Rust port of the cartridge [1], the simulator of that draft, loaded as a Python extension. A branch of it adds what this project needs, all opt-in: our observation computed natively, one-frame decisions, death telemetry, and Turner's per-lane gameplay trace. An earlier version of this draft ran the cartridge's own Lua under a Python PICO-8 runtime; none of its numbers are reused here. Rounds start directly in gameplay with the cartridge's keyboard movement model. All runs here use the hard setting (cartridge difficulty 3) with patterns, powerups, and both enemy types on at 100% spawn rate.

Units. A frame is one 1/60 s update. A decision is one policy action, held for K frames. The current training configuration uses $K = 1$; the discount and GAE decay are defined per decision.

Observation. Each decision sees a single frame (Table I): the player's position and velocity, and a 32×32 grid of 4-pixel cells, each holding an entity category (eight types: three enemy kinds, three pickups, a pattern warning, a lethal pattern rectangle) and that entity's velocity in two planes. Motion is read from the game, not inferred from frame differences, because everything moves less than one cell per decision. When two entities share a cell the more dangerous one is kept. Warnings are encoded for exactly as long as the cartridge draws them. The agent never sees seeds, hidden timers, or future pattern schedules.

Reward. Per frame survived, +0.01; plus the score change, where score from enemies destroying one another (0.5 per enemy, not caused by the agent) is weighted 0.05 and all other score 1.0; and −3.5 on death. These are the weights of the final training run; earlier runs used others (Section VII). A frame cap while the player is alive is a truncation and bootstraps; death is a termination and does not [3].

TABLE I
Policy inputs and outputs.

Item	Content
Player	4 floats: $p_x, p_y, \dot{p}_x, \dot{p}_y$
Grid	32×32 cells: category $\in \{0..8\}$, v_x, v_y
Observation	3,076 floats, one frame, no stack
Field output	$6 \times 32 \times 32$ danger, slices at 4–108 frames
Actions	9 (neutral, 4 cardinal, 4 diagonal)
Value output	1 scalar

IV. Model and Training

Architecture. VelocityFlow v2 (Fig. 1) is fully convolutional up to its output. A trunk of four 3×3 convolutions (32–64 channels, one dilated) maps eleven input planes to 32 feature maps: the category grid expanded to eight one-hot planes, a plane marking the player's own cell, and the two entity-velocity planes. The player's velocity is not among them. Its three heavy layers run at 16×16 and are interpolated back to 32×32. A context branch pools the trunk to 4×4, convolves, and is bilinearly upsampled and concatenated, so every cell sees an arena-wide summary. A 1×1 head emits six danger fields, one per future time $t \in \{4, 12, 24, 48, 72, 108\}$ frames.

Controller. For each action, the cartridge's movement model gives the player's offset at each t (from rest). The controller samples field slice t bilinearly at the player's position plus that offset, weights the six samples by 0.985^t, and negates the sum. Because the offsets start from rest and the trunk sees only the player's cell, nothing on the path from observation to action knows the player's current velocity: a player moving at speed reads danger along paths it cannot follow until its momentum decays, and only the critic receives $\dot{p}_x, \dot{p}_y$. The nine path dangers are standardised across actions and multiplied by a learned temperature to give the logits; there is no policy head. The temperature, initialised at 0.5, grew to 6.4 by update 21,051 and settled at 2.80 in the best checkpoint.

Critic. The value head sees the trunk's channel means, an $8 \times 4 \times 4$ coarse arena map, and the player inputs, through a 64-unit MLP. All activations are LeakyReLU. The model has 111,424 parameters.

PPO. We use Stable-Baselines3 [4] PPO [5] with GAE [6] (Table II). The best checkpoint is update 24,000, after 91.6M decisions. Updates up to about 21,000 collected 256 decisions from each of eight environments; the final run collected 256 from each of 64 (16,384 per update) and ran four epochs of 512-sample minibatches with a KL early stop. Training ran in a dashboard that allowed game settings and reward weights to be changed between updates; checkpoints do not record which settings were in force when, so the training curriculum is not reconstructible from the checkpoints alone.

V. Evaluation Protocol

All numbers are alive frames at 60 Hz on the hard setting with patterns and powerups on. Games are uncapped: each runs until the player dies, as on the training dashboard's

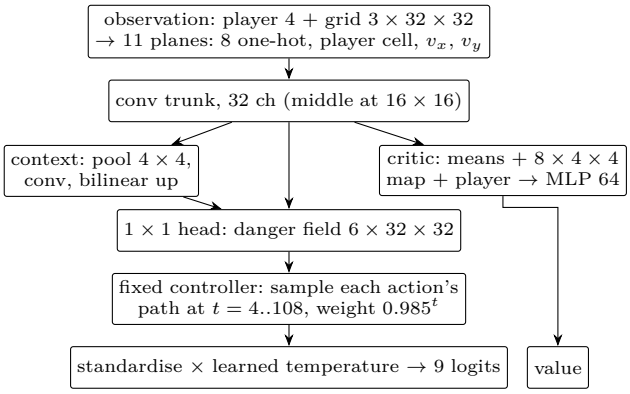


Fig. 1. VelocityFlow v2. The network's only policy output is the danger field; actions are chosen by reading it along simulated paths.

TABLE II
PPO SETTINGS OF THE FINAL TRAINING RUN (UPDATES 21,420–24,000).

Setting	Value
Discount γ, GAE λ	0.997, 0.95
Clip, entropy coef., value coef.	0.2, 0.01, 1.0
Epochs, minibatch, grad-norm clip	4, 512, 0.5
Rollout	64 envs × 256 decisions
Target KL (epoch early stop)	0.02
Learning rate	staged, $3 \times 10^{-4} \rightarrow 3 \times 10^{-5}$

evaluation page, whose code produced every number and chart in Section VI-A onward. For comparison with the earlier draft and [1], we also give means with an 18,000-frame cap. As in [1] we report Worst and Best (single games), Median, Mean, and two quarter means: Q1 is the mean of the worst 25% of games and Q4 the mean of the best 25%; Q1 is not the 25th percentile. We treat the lower tail as primary.

Every condition plays the same 1,024 seeds (128 for the heuristic, which is slow), a contiguous block starting at 1,000,000; training draws seeds outside the reserved evaluation and development ranges. Differences are paired by seed and bootstrapped by seed (2,000 resamples); single-condition intervals use 4,000 resamples for the mean and 1,000 for Q1. Argmax evaluation chooses the most likely action; sampled evaluation draws from the policy. The evaluator runs all 1,024 games in lockstep on the native engine with batched GPU inference. Death telemetry (the killer's class, the pattern active at death, every pattern encounter, and anti-camping triggers) is reported by the engine and does not touch the game's random-number generator.

The best checkpoint was evaluated as tracked in the repository (`velocity-flow-v2-best.zip`, SHA-256 prefix `dcd5f9b10d3913b8`). It was chosen during training on a separate 256-seed development cohort (seeds from 2,000,000), so no result on the evaluation cohort was used to select it or any training setting.

TABLE III
SURVIVAL ON 1,024 EVALUATION SEEDS, HARD SETTING, ALIVE FRAMES, UNCAPPED. 1F/4F: ONE/FOUR FRAMES PER DECISION. Q1/Q4: MEAN OF THE WORST/BEST 25% OF GAMES, NOT PERCENTILES.

Policy	Mode	n	Worst	Q1	Median	Mean	Q4	Best
v2-best	1f argmax	1,024	414	2,902	6,917	8,888	17,887	49,081
v2-best	1f sampled	1,024	170	2,039	5,306	6,070	11,485	33,023
v2-best	4f argmax	1,024	284	991	3,048	3,241	5,800	15,278
v2-best	4f sampled	1,024	125	481	1,485	1,717	3,409	8,591
can-reach-100	1f argmax	1,024	198	951	3,302	3,411	6,117	15,710
random	1f	1,024	99	139	206	225	343	705
gradient field	1f	128	252	254	255	314	465	1,337

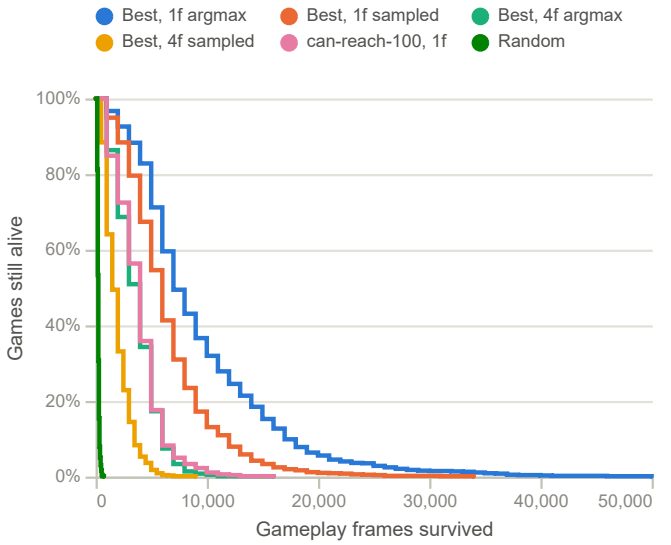


Fig. 2. Share of games still alive at each 500-frame milestone, 1,024 evaluation seeds per line, as drawn by the training dashboard's evaluation page.

VI. RESULTS

A. The best checkpoint

Table III gives the headline numbers. With a decision every frame and argmax actions, the best checkpoint survives a mean of 8,888 frames (95% interval 8,496–9,287), about 2.5 minutes, and a median of 6,917; its worst quarter averages 2,902 (2,658–3,157). 97% of games pass 1,000 frames, 83% pass 4,000 and 43% pass 8,000 (Fig. 2). With no cap, 80 games (7.8%) passed 18,000 frames and the longest lasted 49,081. Capped at 18,000 the mean is 8,355; the checkpoint scored 8,881 on the 256-seed development cohort it was selected on.

On the same 1,024 seeds, the earlier `can-reach-100` checkpoint (5,691 updates) averages 3,411. The best checkpoint is ahead by 5,477 frames in mean (+5,069 to +5,920) and 1,951 in Q1 (+1,696 to +2,222), and outlasts it on 847 of the 1,024 seeds. A uniformly random policy survives 225 frames. The repository's hand-written gradient-field controller survives 314 (128 seeds): on this setting it lingers in the centre, triggers the anti-camping guard in every game, and most games end near frame 255. It is therefore not a meaningful upper reference here, unlike the 6,115-frame heuristic in [1].

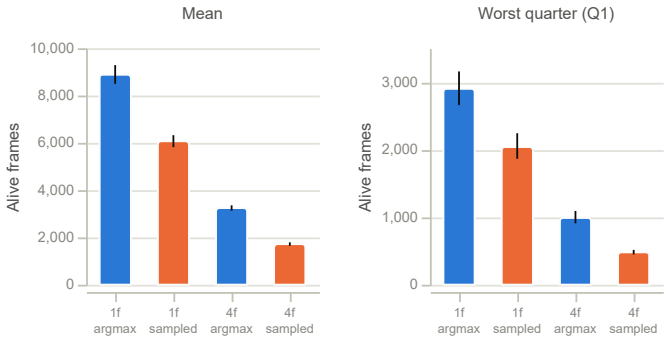


Fig. 3. One frozen checkpoint, four versus one frame per decision, argmax and sampled, on the same 1,024 seeds, with 95% bootstrap intervals. Q1 is the mean of the worst 25% of games.

B. Decision frequency

Turner [1] found that a policy trained with $K = 4$ more than doubled its worst-quarter survival when evaluated at $K = 1$ with frozen weights. We ran the same 2×2 design, $K \in \{1, 4\}$ by action mode, on our checkpoint, whose final training ran at $K = 1$ (Fig. 3). The effect runs the other way and is large. Moving to four-frame decisions lowered the mean by 5,647 frames under argmax (−6,046 to −5,232), 64% of it, and by 4,353 under sampling (−4,614 to −4,116); Q1 fell by 1,911 (−2,184 to −1,643) and 1,558. Under $K = 4$, 14% of games end before frame 1,000, against 3% at $K = 1$, and four-frame play is shorter on 852 of the 1,024 seeds.

Unlike in the earlier draft, argmax is clearly better than sampling at both intervals: by 2,818 frames in mean at $K = 1$ (+2,349 to +3,274) and 863 in Q1 (+560 to +1,148), and by 1,524 in mean at $K = 4$ (+1,382 to +1,661). A likely reason is the learned temperature, which fell from 6.4 at update 21,051 to 2.8 (Section IV): the sampled policy is now much less deterministic, and the cost of its exploratory actions shows.

Read together with [1], the simplest account is still that a policy does best at the decision rate it was trained for. One detail is specific to our controller: its action-path offsets were computed for an action held four frames, and that table is stored in every checkpoint and was not regenerated when training moved to $K = 1$. The policy therefore scores its nine actions as if each were held four frames, while executing each for one.

C. Where games end

Fig. 4 splits the first lethal contact by when the game ended, in windows of the death-time distribution (shortest 10%, the next 30%, the middle 20%, the next 30%, the longest 10%). At $K = 1$, normal enemies account for 60–76% of deaths in every window and 68% overall; kamikazes 12%, static-pattern rectangles 16%, moving-pattern rectangles 4%. Static rectangles matter most in short and middling games: 18% of deaths in the shortest tenth (under 2,600 frames) and 28% between 2,600 and 5,900, falling to 3–4% in games that last past 8,500 frames, where enemies account for about nine deaths in ten. At $K = 4$ the windows are far shorter (the longest tenth starts at 5,700 frames) and static rectangles take 15–27% of deaths in every window.

D. Death telemetry: guard, patterns, and enemies

The engine reports, for every game, the killer's class, the pattern active at death, every pattern encounter by identifier, the frames lived with a pattern active, and every anti-camping trigger. It consumes no random numbers and does not touch reward or observation.

The guard is rarely involved. The anti-camping guard fired in 39 of the 1,024 games at $K = 1$ under argmax, and in 54 of the 4,096 games of the best checkpoint across all four settings. The earlier `can-reach-100` checkpoint triggered it in every one of its 1,024 games. Somewhere between the two checkpoints the policy stopped lingering in the centre.

Patterns are where games end. A pattern is active in 42% of the frames the best checkpoint lives at $K = 1$ but in 69% of its deaths (711 of 1,024): 1.85 deaths per 10,000 frames with a pattern active against 0.60 without, a ratio of 3.1 (Fig. 5). Patterns are deadlier in every window of the game, by 5.4× in the shortest tenth of games and 2.0× in the longest. The ratio shrinks because the death rate with no pattern active rises about eightfold, from 0.14 to 1.16 per 10,000 frames, while the rate with a pattern active only triples: enemy speed and spawn timing scale with score, so between patterns the arena of a long game is far more dangerous than that of a short one. At $K = 4$ the rates are 5.20 and 1.49 (ratio 3.5). The ratio is smaller than the 14–27× reported in [1]. Game length and difficulty plausibly account for part of the gap, since our games are longer and run on the hard setting, but not all of it: even our shortest tenth of games, under 2,600 frames, gives only 5.4×. We have not tested this further.

The policy loses to enemies while patterns are active. 62% of enemy-contact deaths at $K = 1$ (507 of 820) occur during a pattern, and only 204 of the 711 deaths during a pattern are rectangle contact. With 1,024 games each of the 39 patterns in Fig. 6 was met at least 35 times, against a 30-encounter threshold in the earlier draft. Lethality is concentrated but not in a short list: pattern 19 ends 38% of its 98 encounters in death, pattern 35 36% of 144, pattern 13 34% of 82 and pattern 20 33% of 127, and 24 of the 39 end under 10% of encounters in death. At $K = 4$ the same patterns are far deadlier (25: 91%, 20: 84%, 19: 81%, 35: 81%); of the 28 patterns met at least 30 times at both intervals, 24 are more lethal at $K = 4$, and the other four differ by at most 2.5 points.

E. Paths into the deadliest static patterns

Of the 711 deaths during a pattern at $K = 1$ (argmax), 496 happened during a static pattern and 215 during a moving one; 313 games ended with no pattern active. We replayed all 711 pattern deaths with the engine's per-frame gameplay trace, which records the player's path through every pattern, the pattern's rectangles, and how it ended,

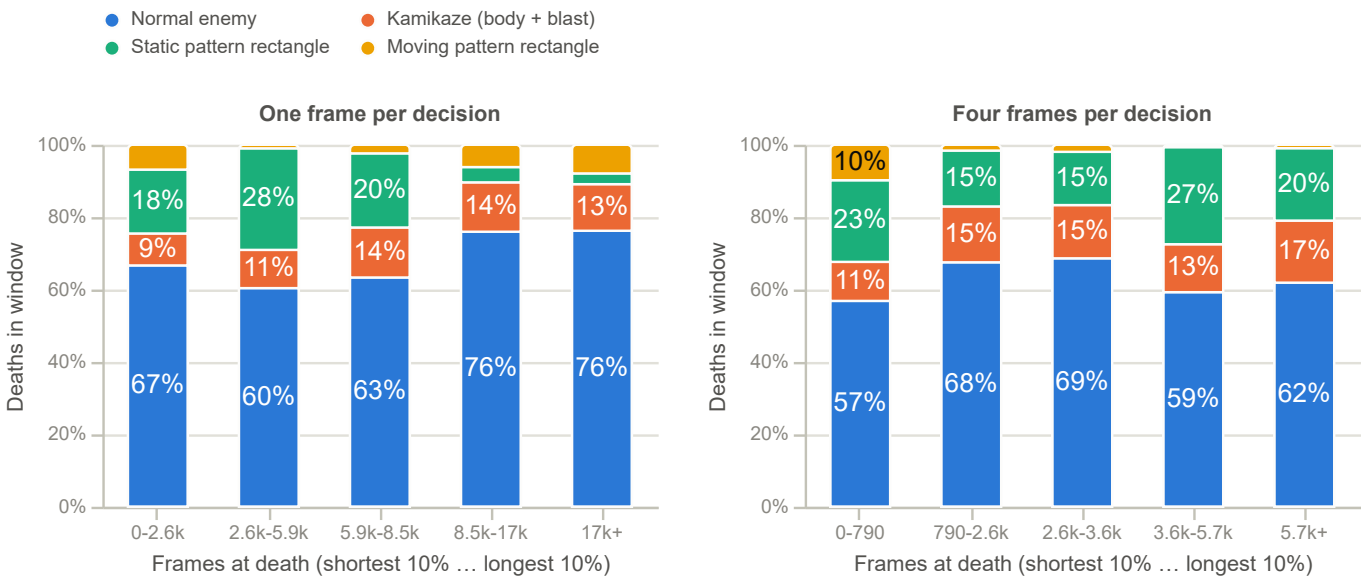


Fig. 4. First lethal contact by the window of the death-time distribution in which the game ended, best checkpoint, argmax, 1,024 games per panel (left $K = 1$, right $K = 4$). Each bar sums to 100%; window boundaries in frames are under the bars.

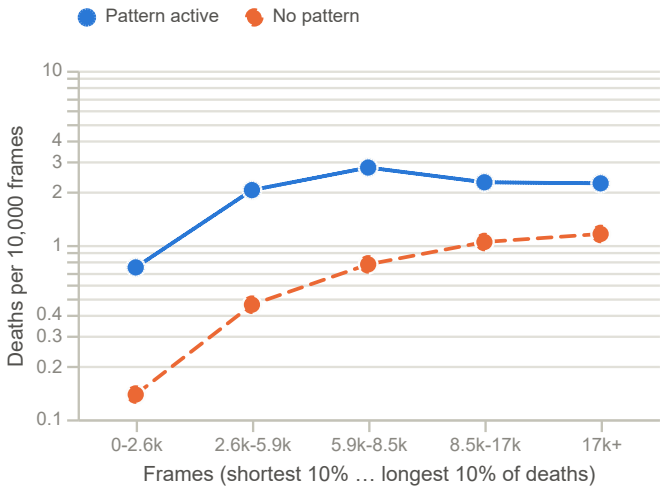


Fig. 5. Deaths per 10,000 frames lived with a pattern active (solid) and without (dashed), by window of the death-time distribution; best checkpoint, $K = 1$, argmax, 1,024 games. Log scale.

as the training dashboard's death-path viewer does. 702 replays ended on the same frame with the same cause as the original game. Nine diverged, ending on a different frame, and are left out of Fig. 7; we attribute them to floating-point differences between batch compositions in GPU inference, since the 711 replays ran as a different batch from the 1,024 originals. The 711 games span 5.87M decisions, so a divergence occurred about once per 650,000 decisions, consistent with rare near-ties between actions that rounding flips and that the game then amplifies. Fig. 7 shows the last 60 frames before death in the four static patterns with the most deaths.

The deadliest layouts kill in different ways. Pattern 24, whose rectangles morph in place (57 deaths), kills mostly through enemies (46): paths end in the open, often low and left of centre, away from the rectangles. In pattern 35, a frame of side and end bars (50 deaths), deaths split almost evenly between the bars (24) and enemies (26); rectangle deaths cluster along the top bar and the upper ends of the side bars, where paths turn along them. Pattern 20, a grid of small blocks (42 deaths), is the one where rectangles dominate (26): paths weave between blocks and end against one. In pattern 34, one large block in a corner with two small ones (39 deaths), 31 deaths are to enemies, clustered just above and left of the large block, where the open space narrows. The shapes of the earlier draft, wall-running into a bar and circling in the open centre, still appear, but at this level of play the commonest end inside a static pattern is an enemy reaching a player whose room to move the layout has cut down.

These are associations at the death frame. They have no counterfactual: we cannot say the policy "should" have turned away. Distinguishing a perception failure from a planning failure needs counterfactual branching: restore a state about 60 frames before death and vary the actions.

VII. Limitations

One lineage. All checkpoints descend from one training lineage; the comparison between checkpoints is between two points of it, not between training replicas, and evaluation seeds do not substitute for those [7], [8]. *Unrecorded training history.* The decision interval, difficulty, reward weights and lives in force during training were changed interactively and are not stored with the checkpoints; the reward described is the final run's configuration. *One evaluation cohort.* A single 1,024-seed cohort gives the reported levels and the checkpoint comparison. The best checkpoint was chosen on a separate development cohort, so its level is not inflated by selection on this one, but there is no third confirmation cohort. *Uncapped games.* Headline

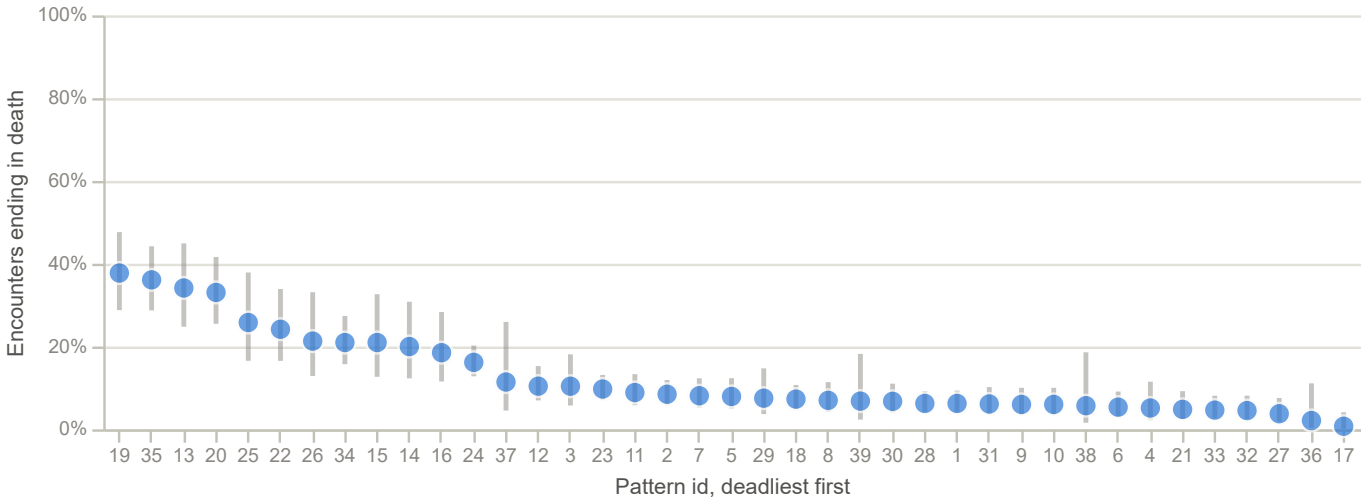


Fig. 6. Share of encounters with each pattern that ended in death, best checkpoint, $K = 1$, argmax, 1,024 games, with 95% Wilson intervals, deadliest first. Score-gated patterns appear only later in a game, so cross-identifier comparisons mix lethality with exposure at higher score.

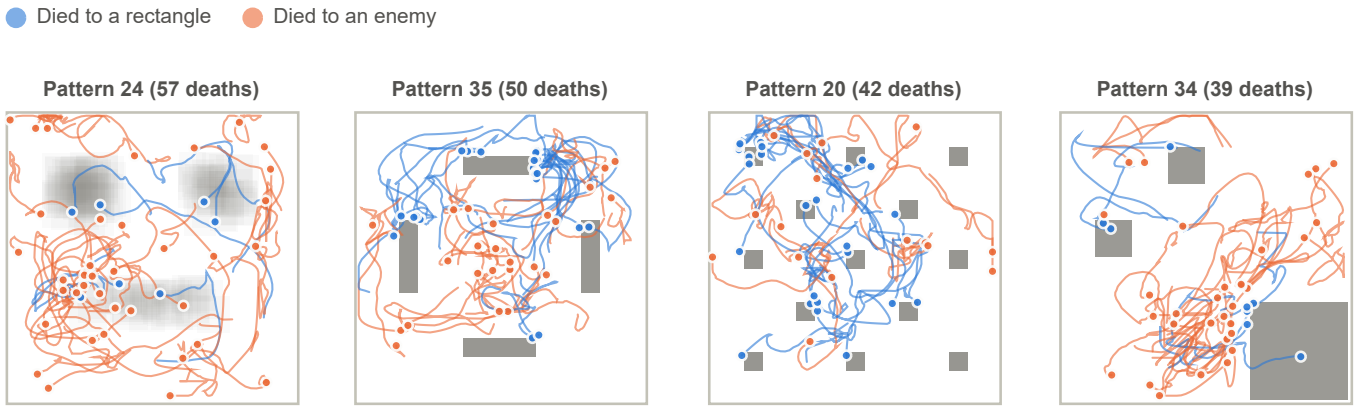


Fig. 7. Last 60 frames before death inside the four static patterns with the most deaths (best checkpoint, $K = 1$, argmax; up to 45 paths each, first by seed; replays that diverged are left out). Blue: died to a rectangle; orange: died to an enemy; dots mark the death position. Grey is the pattern's lethal rectangles at each death frame, overlaid; a pattern whose rectangles morph, such as 24, shows as a blur.

means are uncapped and so carry the very long games: 8% of games pass 18,000 frames, and the capped mean is 8,355. *Replay determinism.* Replays in a different batch reproduced 702 of 711 games exactly; the other nine diverged. *Baselines.* Neither baseline calibrates the upper end: the gradient-field heuristic fails on this setting for a reason unrelated to dodging. *Cross-paper comparison.* Turner [1] reports normal difficulty on the same Rust port with a different reward and observation; we report hard difficulty. Levels are not directly comparable. *No explanation.* The telemetry is descriptive and has no counterfactuals.

VIII. Questions for Practitioners

We would especially value opinions on the following.

1) *Path offsets.* Our controller scores actions with paths for a four-frame hold from rest while executing one-frame decisions from whatever velocity the player has. Should the offsets match the executed interval and start from the current velocity, or is a longer assumed hold a useful bias toward committed motion? How would you test that without retraining from scratch?
2) *Pursuit.* A scalar field over positions cannot express "step aside because it is chasing you"; enemies during patterns are our largest killer. Would you condition the field on the candidate action, add an explicit pursuit channel, or move to a learned head over field features?
3) *Lower tail.* Q1 is a third of the mean. Is the next gain more likely from representation (short-term memory), objective (score in the reward), or exposure (more late-game, high-score patterns in training)?
4) *Selection.* Given one lineage and hours per run, what is the minimum protocol (cohort sizes, confirmation splits, training seeds) you would accept for declaring a checkpoint best?
5) *Baseline.* What hand-written controller would you use as an upper reference on the hard setting, given that a pure danger-gradient follower is defeated by the anti-camping guard?

IX. Conclusion

A field-only PPO controller for Dodge reaches a mean of 8,888 alive frames and a worst-quarter mean of 2,902 on the hardest setting over 1,024 uncapped games, and more training measurably helped: the best checkpoint outlasts an earlier one by 5,477 frames. Decision frequency again matters more than many training updates, but in the direction of the rate the policy was trained for, and argmax now clearly beats sampling. Death telemetry points at enemies that close in while patterns are active, not at the rectangles themselves and not at the anti-camping guard. We offer the setup and numbers as an honest, partly negative record and invite critique.

Availability

The toolchain (environment, policies, training dashboard, and the evaluation and figure scripts) is at https://github.com/LaunchSpark/DodgeAI. It contains no game content: the game runs on Turner's native port, installed separately, and the original cartridge is from [2]. Selected checkpoints are tracked in the repository.

Acknowledgment

Experiments and this draft were prepared with an AI coding agent (Claude) under human direction; the author reviewed the claims and takes responsibility for them. The evaluation protocol follows [1]. The original Dodge game and its art are by Artridge.

References

[1] T. Turner, "An entity-transformer PPO teacher for the Dodge arcade game: Evaluation pitfalls, a control-frequency surprise, and open questions," working draft, 2026.

[2] Artridge, "Dodge," PICO-8 game, Lexaloffle BBS. https://www.lexaloffle.com/bbs/?pid=66443

[3] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018.

[4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[6] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[7] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

[8] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.